



Harbinger: Statistical Tests & ChangeFinder



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Structural Break Tests – Chow and GFT

Ideal for readers looking to connect change-point detection with classical statistical testing ideas, both methods are grounded in linear regression models and offer a principled, hypothesis-driven approach to identifying structural breaks.

Chow

`hcp_chow()` — Tests for structural breaks in linear models using F-statistics evaluated over candidate breakpoint locations. Returns estimated break positions and wraps the `strucchange` package under the hood.

GFT

`hcp_gft()` — Generalized Fluctuation Tests: assesses the stability of regression parameters over time via `strucchange::breakpoints()`, returning break dates selected under information criteria.

Chow Test

The Chow test (1960) detects a single structural break in a linear regression model by splitting the time series at a candidate breakpoint and comparing the fit of two separate regressions against a single pooled regression. A significant difference — measured by an F-statistic — indicates that the model parameters changed at that point.

When to Use

- Series with a single, well-defined structural break
- Piecewise linear behavior (level or slope shift)
- Hypothesis-driven, statistically principled testing

 **Reference:** Chow, G. C. (1960). Tests of equality between sets of coefficients in two linear regressions. *Econometrica*, 28(3), 591–605.

How It Works

01

Fit a linear model to the full series (pooled regression)

02

Split the series at each candidate breakpoint t^*

03

Fit separate models to each segment

04

Compute the F-statistic comparing pooled vs. segmented residuals

05

Reject H_0 of no break if F exceeds the critical value

Chow Structural Break Test

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar e ajustar Chow
model <- hcp_chow()
model <- fit(model, dataset$serie)

# Executar detecção
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 50 TRUE changepoint
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)

## event
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

# Plotar
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step walkthrough

- `hcp_chow()` instantiates the detector with no required parameters
- The method fits a linear model and applies F-statistics over candidate breakpoints
- It correctly identified index **50** as the change-point
- A perfect result: **1 true positive, 0 false positives** on this dataset
- Best suited for series with piecewise linear behavior and a single, well-defined structural break

📌 **Key concept:** The Chow test uses F-statistics to compare models with and without a structural break — directly connecting change-point detection with classical statistical inference.

Generalized Fluctuation Tests

GFT (Zeileis et al., 2002) extends the Chow test by monitoring the stability of regression parameters over the entire time span, rather than testing a single candidate breakpoint. It tracks a fluctuation process derived from recursive or OLS residuals and signals a break when the process exceeds a boundary — allowing detection of multiple breakpoints and gradual shifts.

When to Use

- Series with potentially multiple structural breaks
- Gradual parameter drift rather than a single sharp break
- When model stability over time needs to be formally assessed

Reference: Zeileis, A., Leisch, F., Hornik, K., & Kleiber, C. (2002). strucchange: An R package for testing for structural change in linear regression models. *Journal of Statistical Software*, 7(2), 1–38.

How It Works

01

Fit a linear regression to the full series

02

Compute a fluctuation process (CUSUM or MOSUM) from recursive residuals

03

Compare the process against a critical boundary from functional limit theory

04

Identify breakpoints where the boundary is crossed

05

Select the final set of breaks using information criteria (BIC/AIC)

Generalized Fluctuation Tests

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar e ajustar GFT
model <- hcp_gft()
model <- fit(model, dataset$serie)

# Executar detecção
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 49 TRUE changepoint
## 2 64 TRUE changepoint
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)

## event
## detection TRUE FALSE
##   TRUE  0   2
##   FALSE 1  98

# Plotar
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step by Step

- `hcp_gft()` leverages `strucchange::breakpoints()` internally to perform detection
- It evaluates the stability of regression parameters over time, flagging regions of structural instability
- Breakpoint candidates are selected under information criteria (BIC/AIC), balancing fit and model complexity
- Detected indices **49 and 64** — both turned out to be false positives on this dataset
- GFT tends to be more sensitive than Chow, making it prone to over-detection on series with gradual fluctuations

❏ **Key concept:** GFT assesses the stability of regression parameters over time — particularly useful when the structural break is gradual rather than sudden.

ChangeFinder Variants

Moving beyond classical methods, this family of algorithms detects structural shifts through **predictive surprise**: the model continuously forecasts the next value, and residual deviations are smoothed and thresholded over time to expose underlying change points.

CF-LR

`hcp_cf_lr()` — ChangeFinder with Linear Regression. Tracks residual deviations using a sliding window to capture gradual structural shifts.

CF-ARIMA

`hcp_cf_arima()` — ChangeFinder with ARIMA. Wraps the `forecast` package and applies `harutils()` thresholding to score anomalous residuals.

CF-ETS

`hcp_cf_ets()` — ChangeFinder with ETS. Uses exponential smoothing to model and score residual deviations across the time series.

ChangeFinder with Linear Regression

CF-LR (Yamanishi & Takeuchi, 2002) fits a linear autoregressive model over a sliding window and scores each observation by its prediction error. Scores are smoothed a second time to reduce noise, and a threshold flags change points. The key insight: a structural break surprises the model — residuals spike when the data no longer matches the learned pattern.

When to Use

- Fast, lightweight detection with minimal assumptions
- Series without strong autocorrelation or seasonality
- Exploratory analysis where interpretability matters

Reference: Yamanishi, K., & Takeuchi, J. (2002). A unifying framework for detecting outliers and change points from non-stationary time series data. Proceedings of the 8th ACM SIGKDD, 676–681.

How It Works

01

Slide a window of size `sw_size` over the time series

02

Fit a linear regression model within each window

03

Compute the prediction error (residual) for each observation

04

Apply a second smoothing pass over the residuals

05

Flag observations where the smoothed score exceeds the learned threshold

ChangeFinder with Linear Regression

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar CF-LR
# sw_size controla o tamanho da janela
model <- hcp_cf_lr(sw_size = 10)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## [1] idx event type
## <0 rows> (or 0-length row.names)
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  0  0
##   FALSE 1 100

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step-by-Step Walkthrough

- `sw_size = 10` sets the length of the sliding window used for smoothing
- The model tracks residual deviations and applies a two-stage smoothing process
- No events were detected — 0 true positives and 0 false positives in this run
- The second `har_plot()` displays the residual scores alongside the decision threshold
- Use `attr(detection, "res")` and `attr(detection, "threshold")` to access these values directly

📌 **Key Concept:** ChangeFinder applies two-stage smoothing over model residuals — the detection threshold is learned automatically via `harutils()`, rather than set by hand.

ChangeFinder with ARIMA

CF-ARIMA replaces the linear regression in ChangeFinder with an ARIMA model, capturing temporal autocorrelation and non-stationarity. By modeling the autoregressive and moving-average structure of the data, it produces more accurate forecasts — and therefore more meaningful residuals. A spike in ARIMA residuals signals that the series has deviated from its expected temporal pattern.

When to Use

- Series with significant autocorrelation or temporal dependence
- Non-stationary data with trends or seasonality
- When CF-LR misses breaks due to temporal structure in the data

i Reference: Takeuchi, J., & Yamanishi, K. (2006). A unifying framework for detecting outliers and change points from time series. *IEEE Transactions on Knowledge and Data Engineering*, 18(4), 482–492.

How It Works

01

Slide a window of size `sw_size` over the time series

02

Fit an ARIMA model (via the `forecast` package) within each window

03

Compute one-step-ahead prediction errors (residuals)

04

Smooth residuals with a second-stage moving average

05

Apply a learned threshold (via `harutils`) to flag anomalous observations

ChangeFinder with ARIMA

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar CF-ARIMA
model <- hcp_cf_arima(sw_size = 10)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 51 TRUE anomaly
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  1  99

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step-by-step

- `hcp_cf_arima(sw_size = 10)` uses ARIMA internally via the `forecast` package
- The model flagged index **51** as an **anomaly** — not a **changepoint**
- Result: 0 true positives and 1 false positive — the actual event was missed
- The residuals plot reveals exactly where the score exceeded the learned threshold
- Compare the residuals plot with the series plot to better understand the model's behavior

📌 **Key concept:** ARIMA captures temporal autocorrelation in the residuals — making it better suited for time series with strong temporal dependence than simple linear regression.

ChangeFinder with ETS

CF-ETS substitutes the forecasting model in ChangeFinder with an ETS (Error, Trend, Seasonality) model. ETS uses exponential smoothing to decompose the series into level, trend, and seasonal components, assigning exponentially decaying weights to past observations. When a structural shift occurs, the ETS forecasts lag behind the new regime — producing elevated residuals that drive the change-point score.

When to Use

- Series with smooth trends or seasonal patterns
- Data where level and trend evolve gradually over time
- When ARIMA is too complex and CF-LR is too simple

 **Reference:** Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.
<https://otexts.com/fpp3/>

How It Works

01

Slide a window of size `sw_size` over the time series

02

Fit an ETS model (via the forecast package) within each window

03

Generate one-step-ahead forecasts and compute residuals

04

Smooth residuals with a second-stage moving average

05

Apply a learned threshold (via `harutils`) to flag anomalous observations

ChangeFinder with ETS

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar CF-ETS
model <- hcp_cf_ets(sw_size = 10)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 51 TRUE anomaly
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  0   1
##   FALSE 1  99

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step-by-Step Breakdown

- `hcp_cf_ets(sw_size = 10)` uses Exponential Smoothing (ETS) from the `forecast` package internally
- Like CF-ARIMA, it flagged index **51** as an anomaly rather than a true changepoint
- Results mirror CF-ARIMA: 0 true positives, 1 false positive — the actual event went undetected
- ETS is best suited for series with smooth trends and gradual seasonality patterns
- The residual plot with `yline` clearly visualizes the learned decision threshold

📌 **Key Concept:** ETS models level, trend, and seasonality through exponential smoothing — residuals expose deviations from the model's expected pattern, which drives anomaly scoring.

Volatility & Local Window Methods

These are strong final readings for those who want to go deeper: they focus on more specialized aspects of change detection, such as variance shifts and local window contrasts. They make the most sense after you've built a solid foundation with the earlier methods.

GARCH

`hcp_garch()` — Models conditional variance dynamics. Scores derived from model residuals are smoothed and thresholded to signal shifts in volatility regime.

SCP

`hcp_scp()` — Seminal Change Point: compares linear regression fits with and without the central observation across sliding windows. Large deviations pinpoint the location of a change.

GARCH-Based Change Detection

GARCH (Generalized Autoregressive Conditional Heteroskedasticity) models the conditional variance of a time series rather than its mean. Unlike Chow or CF-LR, which detect shifts in level or trend, GARCH detects volatility regime changes — periods where variance suddenly increases or decreases. This makes it uniquely suited for financial and economic data, where instability often manifests as rising uncertainty rather than a shift in the average value.

When to Use

- Financial or economic series with volatility clustering
- When the mean is stable but variance shifts abruptly
- Series exhibiting heteroskedasticity (non-constant variance)

Reference: Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.

How It Works

01

Fit a GARCH(1,1) model to estimate conditional variance

02

Extract standardized residuals (scaled by conditional std. dev.)

03

Score each observation by its standardized residual magnitude

04

Smooth scores with a moving average to reduce noise

05

Apply a learned threshold to flag volatility regime changes

ChangeFinder with GARCH

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar GARCH-based ChangeFinder
model <- hcp_garch()
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 50 TRUE changepoint
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step-by-step

- `hcp_garch()` models the conditional variance of the time series
- Residual scores are smoothed and thresholded to detect volatility shifts
- Pinpointed exactly index **50** — a perfect result: **1 TP, 0 FP**
- The residuals plot clearly reveals the variance spike at the change point
- GARCH is especially powerful for financial series exhibiting volatility clustering

📌 **Key concept:** GARCH captures regime changes in *variance*, not just in the mean — making it the ideal tool when instability manifests as rising volatility rather than a shift in level.

Seminal Change Point (SCP)

SCP (Ogasawara et al., 2010) detects change points by comparing linear regression fits within a sliding window. For each window position, it fits two models: one using all observations, and one excluding the central observation. A large discrepancy between the two fits — measured by the difference in residual sums — indicates that the central point is structurally different from its neighbors.

When to Use

- Sharp, well-localized structural breaks
- Changes concentrated at a single point rather than spread over a region
- When interpretability and simplicity are priorities

 Ogasawara, E., Martinez, L. C., De Oliveira, D., Zimbrão, G., Pappa, G. L., & Mattoso, M. (2010). Adaptive normalization: A novel data normalization approach for non-stationary time series. *Proceedings of IJCNN*, 1–8.

How It Works

01

Slide a window of size `sw` over the time series

02

Fit a linear regression to all observations in the window

03

Fit a second regression excluding the central observation

04

Compute the score as the difference in residual sums between the two fits

05

Apply a learned threshold to flag windows where the central point is anomalous

Seminal Change Point (SCP)

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar SCP
# sw controla o tamanho da janela (30)
model <- hcp_scp(sw = 30)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 50 TRUE changepoint
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step-by-step

- `hcp_scp(sw = 30)` defines a sliding window of 30 observations
- Compares linear regression fits *with* and *without* the central observation of the window
- Large deviations around the center signal the likely location of a change
- Detected exactly index **50** — a perfect result: **1 TP, 0 FP**
- The window size `sw` controls sensitivity: smaller windows pick up more localized changes

📌 **Key concept:** SCP relies on local contrast — it compares the fit of the full window against the fit without the central point, making it highly sensitive to sharp, well-localized structural breaks.

Summary of Methods

A comparative overview of all change-point detectors available in Harbinger, organized by family and intended use case.

Method	Function	Approach	Best for
AMOC	hcp_amoc()	Cost optimization — single breakpoint	One dominant change point
BinSeg	hcp_binseg(Q)	Recursive binary segmentation	Multiple breakpoints, fast
PELT	hcp_pelt()	Optimal partitioning with pruning	Multiple breakpoints, optimal
Chow	hcp_chow()	F-statistics on linear regression	Breakpoints in linear models
GFT	hcp_gft()	Regression parameter fluctuation	Gradual structural instability
CF-LR	hcp_cf_lr(sw)	Linear regression residuals	Simple predictive surprise
CF-ARIMA	hcp_cf_arima(sw)	Smoothed ARIMA residuals	Series with autocorrelation
CF-ETS	hcp_cf_ets(sw)	Smoothed ETS residuals	Trend and seasonality
GARCH	hcp_garch()	Conditional variance modeling	Volatility shifts
SCP	hcp_scp(sw)	Local window contrast	Sharp, localized breakpoints

Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.

References

Hinkley, D.V. (1970). Inference about the change-point in a sequence of random variables. *Biometrika*, 57(1), 1–17. — Foundational theory underlying AMOC.

Vostrikova, L.Yu. (1981). Detecting "disorder" in multidimensional random processes. *Soviet Mathematics Doklady*, 24, 55–59. — Original formulation of Binary Segmentation.

Killick, R., Fearnhead, P., & Eckley, I.A. (2012). Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500), 1590–1598. — Theoretical basis for BinSeg and PELT.

Zeileis, A., Leisch, F., Hornik, K., & Kleiber, C. (2002, 2003). strucchange: An R package for testing structural change. *Journal of Statistical Software*, 7(2). — Foundation for the Chow and GFT detectors.

Takeuchi, J., & Yamanishi, K. (2006). A unifying framework for detecting outliers and change points from time series. *IEEE Transactions on Knowledge and Data Engineering*. — Theoretical basis for ChangeFinder (LR, ARIMA, and ETS variants).

Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Primary reference for the GARCH and SCP detectors in HARBINGER.